PROBLEM STATEMENT

Software Requirement Specification & Design Rationale for Mini Library Management System

Document: Statement.md **Domain: Information Systems** **Scope: Desktop Application**

1. BACKGROUND & CONTEXT

Small-scale academic institutions, community centers, and personal libraries often rely on manual ledger books or basic spreadsheets to manage their book collections. These traditional methods introduce significant operational inefficiencies, high error rates, and difficulty in real-time inventory tracking.

2. CORE PROBLEM DEFINITION

Traditional manual record-keeping systems suffer from four main vulnerabilities:

1. Tracking Errors
Loss of tracking for borrowed books leads to unreturned materials and unrecorded inventory updates.

2. Inefficient Searching
Searching for specific titles or authors through paper ledgers requires tedious manual scanning.

3. Data Redundancy
Lack of centralized data validation results in duplicate entries and inconsistent status reporting.

4. Poor Accessibility
Non-digital systems lack real-time visibility into whether a book is currently available or issued.

3. PROPOSED SOLUTION

To resolve these inefficiencies, the **Mini Library Management System** provides a lightweight, standalone desktop application designed to centralize inventory, automate issue/return tracking, and enable quick search capabilities through a unified Graphical User Interface (GUI).

4. FUNCTIONAL REQUIREMENTS

MODULE	REQUIREMENT	VALIDATION RULE
Book Entry	Add new book records (Title, Author).	Title and Author fields cannot be blank.
Search	Query records by title or author keywords.	Real-time match using wildcard database queries.
Issue Book	Update status of an available book to 'Issued'.	Prevents issuing if status is already 'Issued'.
Return Book	Update status of an issued book back to 'Available'.	Prevents returning if status is already 'Available'.
Deletion	Remove lost or retired books from storage.	Requires explicit user confirmation before deletion.

> **System Scope**
> The solution prioritizes zero-configuration deployment, utilizing a local embedded database without external server setup, making it immediately operable on standard personal computers.

5. NON-FUNCTIONAL REQUIREMENTS

- **Usability:** Minimalistic single-window GUI requiring zero technical training for library staff.
- **Reliability:** ACID-compliant local database transactions (SQLite3) ensuring data integrity across app sessions.
- **Performance:** Sub-second query responses for catalog browsing and search operations up to thousands of records.
- **Portability:** Cross-platform compatibility (Windows, macOS, Linux) via standard Python standard library modules.

6. KEY USE CASES

USE CASE ID	ACTOR	OBJECTIVE	PRECONDITION
UC-01	Librarian	Catalog a newly acquired book	Application running, title and author provided
UC-02	Librarian	Search for a book requested by a member	Catalog contains active records
UC-03	Librarian	Issue a book to a library member	Book status is set to `Available`
UC-04	Librarian	Process a returned book	Book status is set to `Issued`

7. EXPECTED IMPACT & OUTCOMES

Operational Speed

Reduces book check-out and check-in processing time by over 80% compared to paper logs.

Inventory Accuracy

Eliminates status ambiguity through automated status transitions and error dialog warnings.

8. TECHNOLOGY STACK RATIONALE

- **Python 3:** Chosen for rapid development, readable syntax, and extensive standard library support.
- **Tkinter:** Native Python GUI library providing lightweight desktop UI components with zero external dependencies.
- **SQLite3:** Serverless, file-based relational database management system delivering lightweight persistence without configuration overhead.